PRD: Rainbow Reveal (working title)

js13kGames 2026 — Theme: Unicorns and Rainbows

1. One-line pitch

Steer a unicorn across a dark canvas — its trail is the only light. Uncover a hidden picture with the fewest possible strokes, then type what you saw before the reveal fades.

2. Goals & non-goals

Goals

Ship a complete, playable, submittable game **well under 13,312 bytes zipped**.
Core loop must be understandable in under 10 seconds with no tutorial text.
Procedural art only — zero image/audio asset files.
Compete in **Online** category (required if Wavedash leaderboards/achievements are used) in addition to Desktop/Mobile base category, once confirmed eligible under Online's specific rules.

Non-goals (cut for size/time)

No level editor.
No multiple hidden-image "packs" beyond a small built-in set (procedurally generated or hand-authored as compact path data).
No mobile-specific control scheme beyond touch-drag parity with mouse — no separate UI.
No sound (cut first if size is tight; see §7).

3. Theme interpretation

Rainbow = the trail/light mechanic itself — a gradient stroke behind the unicorn, replacing a plain "fog of war" wipe.
Unicorn = the player-controlled cursor/sprite, drawn as a tiny procedural silhouette (triangle body + horn line), not a raster sprite.
Hidden images to guess are themselves unicorn/rainbow-adjacent (a rainbow, a horn, a cloud, a star, a horseshoe) — keeps word list short and on-theme.

4. Core gameplay loop

Round starts: canvas is black. A hidden image is loaded (vector/path-based, drawn off-screen to a mask).
Player moves the unicorn (mouse / touch-drag / arrow keys) around the canvas.
Anywhere the unicorn's trail passes, a rainbow-colored circular "reveal" is punched into the mask, permanently showing the underlying image in that area.
A **move counter** ticks based on distance traveled (not time) — this is the score driver.
Player can guess at any time by pressing Enter / tapping a "Guess" button, which opens a single text input.
Correct guess → next image loads immediately (campaign advances). Moves-for-this-round are locked in and added to the running total.
Incorrect guess → **lose 1 life** (player starts with **3 lives total**), input shakes, mask/canvas resets or stays as-is (TBD, see §9), same image continues — no move penalty on top of the life loss, keeps the two costs (lives vs. moves) distinct and legible.
Game over when lives reach 0 → final score = total moves across all rounds completed → submit to Wavedash leaderboard.

Campaign complete (all images guessed) → final score = total moves across the full set → submit to Wavedash leaderboard, lives remaining shown as a bonus/flourish but not scored (keep scoring single-dimensional: moves only).
Win condition (per round): correct guess → advance. **Loss condition (run):** 3 wrong guesses across the whole run → game over. **Score:** total moves (distance) across the run — lower is better, ideal for a leaderboard. Runs that end via game-over vs. full-campaign-clear should probably be flagged separately on the leaderboard (see §9) so a lucky low-move game-over doesn't outrank a full clear.

5. Controls

Desktop: mouse move = steer (unicorn follows cursor with slight lag or 1:1 — TBD in prototyping), keyboard arrows as alt input, Enter = open guess box.
Mobile: touch-drag = steer, tap a small "?" button = open guess box (must stay in the 13KB text-input footprint — use a native `<input>` overlay, not a custom canvas keyboard).
No settings menu. No pause. Restart via a single visible "↻" icon.

6. Technical architecture

Rendering: single `<canvas>`, 2D context. No WebGL (not needed, and WebGL boilerplate costs bytes).
Reveal technique (the core trick):
Draw the hidden image to an **offscreen canvas** once per round (procedural vector paths — lines/circles/beziers — not bitmap data).
Maintain a second canvas as the "mask": start filled black.
On movement, use `globalCompositeOperation = 'destination-out'` with a soft circular gradient brush to punch transparent holes in the mask at the trail position, revealing the image canvas beneath.
Composite: image canvas → mask canvas (with holes) → drawn on top each frame. This avoids needing to touch pixel data directly (cheap on CPU, cheap on code size — no manual `ImageData` loops needed for the reveal itself).
Rainbow trail: draw a thin `hsl()` gradient stroke along the recent path, hue cycling with distance traveled — purely decorative, layered above the mask.
Hidden images: authored as compact arrays of draw commands (e.g. `[['M',x,y],['L',x,y],['C',...]]` or simplified custom opcodes) — NOT SVG files, NOT PNGs. Keep each image's data under ~300–500 bytes. This is the single biggest size risk — budget and prototype this first.
Move counter: accumulate Euclidean distance between successive input positions each frame; convert to an integer "moves" score via a scale constant (tune during playtesting, e.g. 1 move per 10px).
Guess input: a real HTML `<input>` element positioned absolutely over the canvas, hidden by default, shown on guess-trigger. Reuse browser text rendering/keyboard instead of drawing one — this is free functionality, don't reinvent it.
Wavedash integration: isolated in one small module; only called on round-complete (submit score) and on menu load (fetch leaderboard/achievements). Must degrade gracefully (game still fully playable) if the call fails or is blocked — required since Wavedash is a network dependency and cannot be a hard requirement for core gameplay per the "external resources" rule.

7. Size budget (target: ≤ 13,312 bytes zipped)

Piece	Rough budget
index.html + inline JS (minified)	~7,000–8,500 B
Hidden image path data (5 images)	~1,500–2,500 B
Inline CSS (minimal, embedded in HTML)	~300–500 B
Wavedash client snippet (if bundled, not CDN)	~500–1,500 B (verify actual footprint before committing)
Buffer / slack	~1,000 B minimum, always reserve this

Cut order if over budget (cut top-down until it fits):
Sound (never budgeted in — cut candidate #0, skip entirely unless size allows)
Reduce hidden-image count from 5 → 3
Simplify rainbow trail (flat color instead of hue-cycling gradient)
Drop campaign/multi-round structure → single endless mode, one code path
Trim Wavedash to leaderboard-only, cut achievements
Tooling: minify JS (esbuild/terser), inline everything into one HTML file, zip with max compression (`zip -9`). Check actual zipped size continuously during dev, not just at the end — don't discover you're 4KB over on submission day.

8. Category & compliance checklist

- [ ] Confirm Online category's specific extra rules (not in the base rules doc provided) before finalizing Wavedash use — base rules only confirm that external network reliance excludes a game from the *overall* ranking unless the category permits it.
- [ ] Game must work fully offline / with Wavedash unreachable (no console errors either way — hard requirement, checked in both Chrome and Firefox).
- [ ] All hidden-image path data and code is original or rights-cleared (no traced copyrighted art).
- [ ] If using `localStorage` for anything client-side (e.g. caching a name), prefix keys with a unique namespace per the "be neighbourly" rule.
- [ ] GitHub repo has full buildable source (not just the minified bundle) — set this up from day one, not at the end.
- [ ] Zero external asset/network requirement for base gameplay (fonts, images, analytics all forbidden per FAQ).

9. Open questions (need a decision before/during build)

~~Structure~~ **Decided:** fixed campaign (5 images), 3 shared lives, one running move-total score. Correct guess → next image; wrong guess → lose a life; 0 lives or campaign clear both end the run and submit a score.
Wavedash scope: leaderboard only, or also achievements (e.g. "cleared the run", "cleared with 3 lives left", "under X total moves")? Achievements add code + data size — with a lives system, "no-death clear" is a natural cheap achievement to include if budget allows.
Cursor-follow feel: 1:1 cursor tracking vs. slight lag/inertia on the unicorn — affects "juice" but also affects how movement maps to the move-counter (needs to feel fair, not punishing for overshoot).
~~Wrong-guess penalty~~ **Decided:** no move penalty on a wrong guess — cost is a life instead. Need to decide: does the mask/reveal progress on the current image reset on a wrong guess, or persist (letting the player

keep the partial reveal and try again)? Persisting is more forgiving and probably better for a low-pressure feel; resetting raises difficulty but adds replay-the-same-image tedium risk.
Word list difficulty: free-text match (with fuzzy/case-insensitive compare) vs. multiple choice — free text is more satisfying but needs a small synonym-tolerant matcher, which costs bytes.
Leaderboard integrity: should a game-over run (fewer images cleared) be directly comparable on the same leaderboard as a full clear? A partial run will almost always have a lower move-count than a full clear, which would make incomplete runs rank artificially higher. Consider either (a) only submitting scores for full clears, or (b) tagging/sorting full clears above partial runs regardless of move count.

10. Milestones (mapped to jam deadlines)

Milestone	Target
Prototype: mask-reveal technique + move counter working, no art polish	Week 1
Vector hidden-image authoring pipeline + 3 test images	Week 1–2
Guess input + win/lose flow, full single-round loop playable	Week 2
Wavedash integration (leaderboard submit + fetch), degrade-gracefully tested	Week 2–3
Size pass: minify, inline, zip, trim per §7 cut order until under budget	Week 3
Cross-browser check (Chrome + Firefox, console-error-free)	Week 3
Repo cleanup for GitHub submission (readable source, README)	Before 13 Sept 13:00 CEST
Submit	By 13 Sept 13:00 CEST deadline

(Weeks are relative to your own start date — align against the actual submission window: 13 Aug–13 Sept 2026.)

11. Success criteria

Zipped build ⩽ 13,312 bytes, unzips and runs standalone with `index.html` at top level.
No console errors in Chrome or Firefox during a full playthrough.
A first-time player can complete one round without instructions.
Wavedash leaderboard/achievements work when reachable and fail silently (no crash, no console error) when not.